LABORATORY OBSERVATION

Course:	Full Stack Web Development	**Experiment No.:**	6 (Exp 3)
Course Code:	23CM4121	**Date:**	05-10-2026
Student Name:	Nimidithalli Nandu	**Roll No.:**	A24126552100

1. TITLE

Package Management and Dependency Configuration Using NPM and package.json

2. AIM

To understand Node Package Manager (NPM), project initialization, manifest configuration via package.json, and dependency installation for an Express web application.

3. OBJECTIVE

- Initialize and configure a Node.js project manifest using package.json.
- Configure metadata, scripts, and dependencies (express, chalk).
- Verify dependency locking and version specifications with package-lock.json.
- Implement and run a lightweight Express web server on port 3000.

4. THEORY

NPM (Node Package Manager) is the official dependency manager for Node.js. It facilitates installing, sharing, and managing external modules and libraries.

package.json is the core manifest file of every Node.js project. It describes the project title, version, author, licensing, scripts, and runtime dependencies.

package-lock.json records the exact versions of dependencies installed in node_modules, ensuring deterministic and reproducible builds across environments.

5. ALGORITHM / PROCEDURE

1. Initialize package.json defining name, version, author (Nimidithalli Nandu), and dependencies.

2. Declare express and chalk dependencies within the dependencies section.

3. Create code.js and import the express module.

4. Define a root GET route handler returning "Hello from Express!".

5. Call app.listen(3000) to start the web server and print confirmation to console.

6. PROGRAM

package.json

```
{
  "name": "npm_json",
  "version": "1.0.0",
  "description": "NPM and package.json demonstration with Express and dependencies",
  "main": "code.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node code.js"
  },
  "keywords": [
    "npm",
    "package.json",
    "express"
  ],
  "author": "Nimidithalli Nandu",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "chalk": "^4.1.2",
    "express": "^5.2.1"
  }
}
```

code.js

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
    res.send("Hello from Express!");
});

app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```

7. CODE EXPLANATION

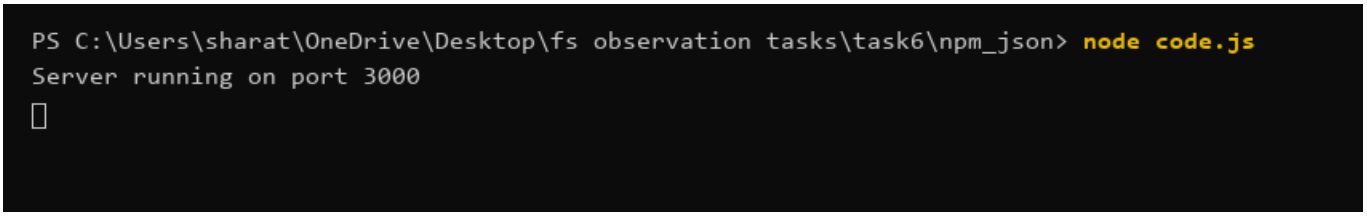
Code Element	Detailed Explanation
package.json	Project manifest storing package metadata, scripts, and dependency version bounds.
dependencies: { express, chalk }	Declares external production packages required by the application.
const app = express()	Creates a web server instance using the installed Express dependency.
app.get("/", ...)	Registers an HTTP GET route returning a text greeting.
app.listen(3000, ...)	Binds the HTTP server to listen on port 3000.

8. TEST CASES AND EXECUTION DATA

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Read package.json manifest	Validate JSON syntax and dependencies	Manifest valid with author and dependencies	**Pass**
TC-02	Verify package-lock.json	Ensure exact dependency version hashes	Dependencies locked accurately	**Pass**
TC-03	Execute code.js	Server starts listening on port 3000	Console logs "Server running on port 3000"	**Pass**

9. OUTPUT

Terminal Output of npm_json/code.js Server



10. RESULT

NPM package management, manifest configuration via package.json, and Express server dependency execution were successfully demonstrated.